



# APPLIED MATH FOR COMPUTER SCIENCE

## CPCS 212

LAB 4 : MATLAB programming

# Agenda

---

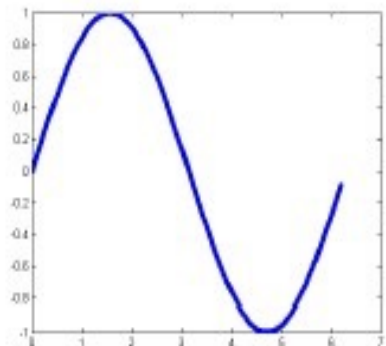
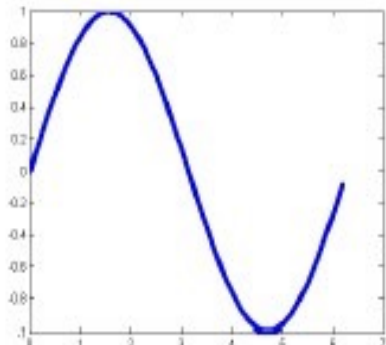
- User Defined Functions
- Relational operators
- Logical operators
- Program Control Statements

# Lab 4

---

Open the diary file  
Lab4\_yourname.txt

# Script versus functions in Matlab

|          |                                                                                                                   |                                                                                            |                                                                                      |
|----------|-------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Script   | Doesn't start by keyword "function"                                                                               | 1. Open Matlab Editor and write the following statements:                                  |   |
|          | Can be saved using any valid name                                                                                 | <pre>x=0:0.1:2*pi;<br/>y=sin(x);<br/>plot(x,y,'LineWidth',5)</pre>                         |                                                                                      |
|          | Doesn't accept input and output arguments                                                                         |                                                                                            |                                                                                      |
|          | Variables are shared with WS (Global)                                                                             | 2. Save the file with name "mys.m"                                                         |                                                                                      |
|          | Can be called by typing its name on WS                                                                            | 3. Type <code>mys</code> on WS and press <code>Enter</code>                                |                                                                                      |
| Function | Must start by keyword "function"<br><code>function [outArgs] = funName(inArgs)</code>                             | 1. Open Matlab Editor and write the following statements:                                  |  |
|          | Must be saved using function name <code>funName.m</code>                                                          | <pre>function myf()<br/><br/>x=0:0.1:2*pi;<br/>y=sin(x);<br/>plot(x,y,'LineWidth',5)</pre> |                                                                                      |
|          | Accepts input and/or output arguments <code>inArgs</code> and <code>outArgs</code>                                |                                                                                            |                                                                                      |
|          | Variables are not shared with WS (Local)                                                                          | 2. Save the file with name "myf.m"                                                         |                                                                                      |
|          | Can be called by typing its name on WS and following the function header <code>[outArgs] = funName(inArgs)</code> | 3. Type <code>myf()</code> on WS and press <code>Enter</code>                              |                                                                                      |

4

# Function arguments

|                  |                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                |
|------------------|-------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Input Arguments  | Local variable names                                                                                              | <pre>function [x,y]=myf1(a,b)  fprintf('you have used %d input arguments\n',nargin); fprintf('you have used %d output arguments\n',nargout); if nargin==1     if isempty(inputname(1))         fprintf('you have used direct value in input argument #%d\n',1);     else         fprintf('you have used %s in input argument #%d\n',inputname(1),1);     end elseif nargin==2     for i=1:2         if isempty(inputname(i))             fprintf('you have used direct value in input argument #%d\n',i);         else             fprintf('you have used %s in input argument #%d\n',inputname(i),i);         end     end end</pre> |                                                                                                                                                                |
|                  | Used to <b>receive values</b> from caller when calling the function                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                |
|                  | Caller can call using <b>at most all input arguments</b> depending on <b>function code</b>                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                |
|                  | <b>nargin</b> : number of <b>input arguments</b> that were <b>used to call</b> the function                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                |
|                  | <b>inputname(n)</b> : caller's workspace <b>variable name</b> corresponding to the input argument number <b>n</b> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                |
| Output Arguments | Local variable names                                                                                              | >> myf1()                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | you have used 0 input arguments<br>you have used 0 output arguments                                                                                            |
|                  | Used to <b>return values</b> to caller after calling the function                                                 | >> x=2;<br>>> b= myf1(1,x)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | you have used 2 input arguments<br>you have used 1 output arguments<br>you have used direct value in input argument #1<br>you have used x in input argument #2 |
|                  | Caller can call using <b>at most all output arguments</b> depending on <b>desired return values</b>               | >> [a,b]=myf1(1,2,3)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | ??? Error using ==> myf1<br>Too many input arguments.                                                                                                          |
|                  | <b>nargout</b> : number of <b>output arguments</b> that were <b>used to call</b> the function                     | >> [a,b,c]=myf1(1,2)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | ??? Error using ==> myf1<br>Too many output arguments.                                                                                                         |

# Variable scope

|                         |                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                     |                                              |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| local                   | <ul style="list-style-type: none"> <li>- All function <b>variables</b> are <b>local</b> unless they declared global or persistent</li> <li>- Created when calling the function and <b>cleared</b> from memory after returning to caller</li> </ul>                                                                                                        | <pre>function y=myf2(x)  global a persistent b  if isempty(a)     a=0; end if isempty(b)     b=1; end  a=a+1; b=b+1; y=x+a+b;</pre> |                                              |
| global <b>a b c</b>     | <ul style="list-style-type: none"> <li>- Declares <b>a b c</b> as global variables</li> <li>- All functions or <b>WS</b> declare a variable as global will <b>share</b> this variable</li> </ul>                                                                                                                                                          |                                                                                                                                     |                                              |
| persistent <b>a b c</b> | <ul style="list-style-type: none"> <li>- Declares <b>a b c</b> as persistent variables (<b>local</b> variables)</li> <li>- A function declare a variable as persistent will <b>retain its value</b> between calls to this function</li> <li>- To initialize a persistent variable <b>a</b>:<br/> <pre>if isempty(a)     a=init_val; end</pre> </li> </ul> | <pre>&gt;&gt; myf2(1) &gt;&gt; myf2(1) &gt;&gt; global a &gt;&gt; a=5; &gt;&gt; myf2(1) &gt;&gt; myf2(1)</pre>                      | <pre>ans = 4 ans = 6 ans = 11 ans = 13</pre> |

# User Defined Functions

- Functions are m-files which can be executed by specifying some inputs and supply some desired outputs.
- The code is telling the Matlab that an m-file is actually a function
- The first line in a function file must begin with a function definition line that has a list of inputs and outputs.
- This line distinguishes a function M-file from a script M-file. It's syntax is as follows:

**function** [output variables]= name (input variables)

## Note:

- The output variable(s) are enclosed in square brackets [ ], while the input variable(s) must be enclosed with parentheses ( ).
- The function name : name should be the same as the file name in which it is saved (with the .m extension).

# Examples of Function Definition Lines

---

## 1. One input, one output:

```
function [area_square] = square(side)
```

## 2. Brackets are optional for one output:

```
function area_square = square(side)
```

## 3. Three inputs, one output:

```
function [volume_box] = box(height, width, length)
```



# Examples of Function Definition Lines

## 4. One Input, two outputs:

```
function [area_circle ,circumf] =circle(radius)
```

## 5. No Input, No output:

```
function show_message
```

```
1 function show_message
2     s='welcome in MATLAB'
3     end
```

```
>> show_message
```

```
s =
```

```
welcome in MATLAB
```

# Examples of Function Definition Lines

**Examples:** *Demonstrate use of I/O arguments*

**twosum.m**

```
function twosum(x,y)
% twosum  Add two matrices
%         and print the result
x+y
```

two inputs, no output

**threesum.m**

```
function s = threesum(x,y,z)
% threesum  Add three variables
%           and return the result
s = x+y+z;
```

three inputs, one output

**addmult.m**

```
function [s,p] = addmult(x,y)
% addmult  Compute sum and product
%          of two matrices
s = x+y;
p = x*y;
```

two inputs, two outputs

# Example of number of input arguments

---

```
function c = addme(a, b)
switch nargin
case 2
c = a + b;
case 1
c = a + a;
otherwise
c=0;
end
```

# Program Control Statements in Matlab

---

- Matlab uses decision and loop structures to control program execution.
- The following structures are available in Matlab:
  - **if statements**
  - **for loops**
  - **while loops**
  - **switch statements**
  - **break statements**
  - **continue statements.**

# Program Control Statements in Matlab

|                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                                                                                       |                   |
|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| <pre>if logical_expression     statements end</pre>                                                                       | <ol style="list-style-type: none"> <li>1. Evaluates <code>logical_expression</code></li> <li>2. <b>True</b>: Executes <code>statements</code></li> </ol>                                                                                                                                                                                                                                                                                                                      | <pre>&gt;&gt; if 2+4&gt;1+2     a=5 end</pre>                                                                                         | <pre>a = 5</pre>  |
| <pre>if logical_expression     statements1 else     statements2 end</pre>                                                 | <ol style="list-style-type: none"> <li>1. Evaluates <code>logical_expression</code></li> <li>2. <b>True</b>: Executes <code>statements1</code> → end</li> <li>3. <b>False</b>: Executes <code>Statements2</code> → end</li> </ol>                                                                                                                                                                                                                                             | <pre>&gt;&gt; if 2+4&lt;1+2     a=5 else     a=0 end</pre>                                                                            | <pre>a = 0</pre>  |
| <pre>if logical_expression1     statements1 elseif logical_expression2     statements2 else     statements3 end</pre>     | <ol style="list-style-type: none"> <li>1. Evaluates <code>logical_expression1</code></li> <li>2. <b>True</b>: Executes <code>statements1</code> → end</li> <li>3. <b>False</b>: Evaluates <code>logical_expression2</code></li> <li>4. <b>True</b>: Executes <code>statements2</code> → end</li> <li>5. <b>False</b>: Executes <code>statements3</code> → end</li> </ol>                                                                                                      | <pre>&gt;&gt; if 2&gt;4     a=5 elseif 2&lt;1     a=6 else     a=10 end</pre>                                                         | <pre>a = 10</pre> |
| <pre>switch expression case value1     statements1 case value2     statements2 . . . otherwise     statements_n end</pre> | <ol style="list-style-type: none"> <li>1. Evaluates <code>expression</code> (<b>result</b> must be scalar or string)</li> <li>2. If <b>result</b> matches <code>value1</code>, Executes <code>statements1</code> → end</li> <li>3. If <b>result</b> matches <code>value2</code>, Executes <code>statements2</code> → end</li> <li>4. .</li> <li>5. .</li> <li>. .</li> <li>n. . If <b>result</b> doesn't match any value, Executes <code>statements_n</code> → end</li> </ol> | <pre>&gt;&gt; switch 1+2-4*5     case 1         a=1     case 2         a=2     case 3         a=3     otherwise         a=0 end</pre> | <pre>a = 0</pre>  |

|                                                |                                                                                                                                                                                                                                                |                                                                                                                                                                    |                                          |
|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| <pre>for i = a:b:c     statements end</pre>    | <ol style="list-style-type: none"> <li>1. Puts <code>i=a</code></li> <li>2. Tests <code>i&lt;=c</code></li> <li>3. <b>True</b>: Executes <code>statements</code> → Puts <code>i=a+b</code> → Go to 2</li> <li>4. <b>False</b> → end</li> </ol> | <pre>&gt;&gt; for i=1:3     disp(i) end</pre>                                                                                                                      | <pre>1 2 3</pre>                         |
| <pre>while expression     statements end</pre> | <ol style="list-style-type: none"> <li>1. Evaluates <code>expression</code></li> <li>2. <b>True</b>: Executes <code>statements</code> → Go to 1</li> <li>3. <b>False</b> → end</li> </ol>                                                      | <pre>&gt;&gt; i=5; &gt;&gt; while i&gt;0     i=i-1 end</pre>                                                                                                       | <pre>i = 4 i = 3 i = 2 i = 1 i = 0</pre> |
| <code>continue</code>                          | <p><b>Passes</b> control to the <b>next iteration</b> of the <b>for</b> or <b>while</b> loop in which it appears, skipping any remaining statements in the body of the loop</p>                                                                | <pre>&gt;&gt; i=10; &gt;&gt; while i&gt;0     i=i-1     if i&lt;6         if i==2             break         end     end     i=i-2     continue end i=i-3 end</pre> | <pre>i = 9 i = 6 i = 5 i = 3 i = 2</pre> |
| <code>break</code>                             | <p><b>Terminates</b> the execution of a <b>for</b> loop or <b>while</b> loop</p>                                                                                                                                                               |                                                                                                                                                                    |                                          |

# Example 1- while

- *Write out the values of  $x^2$  for all positive integer values  $x$  such that  $x^3 < 2000$ .*

```
x = 1;  
while x^3 < 2000  
    disp(x^2)  
    x = x+1;  
end
```

# Example 2 - switch/case/otherwise

```
y = [12 64 24];  
plottype = input('Enter the plot type','s')  
switch plottype  
    case 'bar'  
        bar(y)  
        title('Bar Graph')  
    case {'pie','pie3'}  
        pie3(y)  
        title('Pie Chart')  
    case {'Plot'}  
        plot(y)  
        title('Plot Chart')  
    otherwise  
        warning('Unexpected plot type. No plot created.')  
end
```



## Example 3 – for/break

```
for k=-10:1:10
    if (k^2-50<0)
        break;
    end
    val=k^2-50;
    fprintf('k=%d  val=%d\n',k,val)
end
```

# Example 4 – try-catch

- **Checking for Errors with try-catch**

```
function matrixMultiply(A, B)
try
    X = A * B
catch
    disp '** Error multiplying A * B'
end
```

**Input 1:**

```
A = [1 2 ; 6 7 ; 0 1 ];    B = [9 5 6; 0 4 9];
```

**Input 2:**

```
A = [1 2 3; 6 7 2; 0 1 5];    B = [9 5 6; 0 4 9];
```

# Relational operators

- Relational operators always operate element-by-element.
- The result of applying relational operators is a logical value (i.e. the result is either true or false)

| Operators | Description           | Example                                               | Output |   |   |   |
|-----------|-----------------------|-------------------------------------------------------|--------|---|---|---|
| ==        | Equal                 | >> X= [6 4 9 10];<br>>> Y= [3 5 9 11];<br>>> Z=(X==Y) | Z = 0  | 0 | 1 | 0 |
| ~=        | Not equal             | >> Z=(X~=Y)                                           | Z = 1  | 1 | 0 | 1 |
| <         | Less than             | >> Z=(X<Y)                                            | Z = 0  | 1 | 0 | 1 |
| >         | Greater than          | >> Z=(X>Y)                                            | Z = 1  | 0 | 0 | 0 |
| <=        | Less than or equal    | >> Z=(X<=Y)                                           | Z = 0  | 1 | 1 | 1 |
| >=        | Greater than or equal | >> Z=(X>=Y)                                           | Z = 1  | 0 | 1 | 0 |

# Logical operators.

| Operator | Description               | Example                                              | output  |   |   |   |   |
|----------|---------------------------|------------------------------------------------------|---------|---|---|---|---|
| &&       | Short-circuit logical AND | >> a=1;b=0;<br>>> a&& b                              | ans = 0 |   |   |   |   |
|          | Short-circuit logical OR  | >> a=1;b=0;<br>>> a   b                              | ans = 1 |   |   |   |   |
| &        | Element-wise logical AND  | >> A = [ 0 1 1 0 1]<br>>> B = [ 1 1 0 0 1]<br>>> A&B | ans = 0 | 1 | 0 | 0 | 1 |
|          | Element-wise logical OR   | >> A B                                               | ans = 1 | 1 | 1 | 0 | 1 |
| ~        | Logical NOT               | >> ~A                                                | ans = 1 | 0 | 0 | 1 | 0 |

# Find Function

- The find function determines the indices of all nonzero elements in a logical matrix

```
>> A = magic(4)
```

```
A =
```

|    |    |    |    |
|----|----|----|----|
| 16 | 2  | 3  | 13 |
| 5  | 11 | 10 | 8  |
| 9  | 7  | 6  | 12 |
| 4  | 14 | 15 | 1  |

```
>> i = find(A > 8)
```

```
i =
```

```
1  
3  
6  
8  
10  
12  
13  
15
```



# Examples of Relational Operators

- **Note:** the difference between the result obtained by  $x$  ( $x \sim= 0$ ) and the result obtained by **find**( $x$ ).

```
>> x = [-2, 0, 4];  
>> y = find(x)
```

Index

```
y =
```

1

3

```
>> y = x(x ~ = 0)
```

```
y =
```

-2

4

Value

```
>> y = x ~ = 0
```

```
y =
```

1

0

1

Logic matrix

# Assignment 4-1

- Write a Matlab function that computes and display the surface area, and the volume of a cube, given its side length as an input argument. Your function should set the output arguments to 0 for a non-positive side length. Do not forget to include proper comments.
- Write the proper Matlab statement(s) to compute and display the surface areas and volumes for cubes with side length 5.
- **Note:**
- Surface area of a cube =  $\text{side length} * \text{side length}$
- Volume of a cube =  $\text{side length} * \text{side length} * \text{side length}$

# Assignment 4-2

The capacitance of two parallel conductors of length  $L$  and radius  $r$ , separated by a distance  $d$  in air, is given by

$$C = \frac{\pi \epsilon L}{\ln \left( \frac{d}{r} \right)}$$

Where  $\epsilon$  is the permittivity of air ( $\epsilon = 8.854 \times 10^{-12}$  F/m).

- Write a script file that accepts user input for  $d$ ,  $L$ , and  $r$ , then computes and displays  $C$ . The screen dialogue should appear as follows:

```
Enter the length of conductor:  
Enter the radius of conductor:  
Enter the distance:  
The capacitance of two parallel conductors is:
```



# Assignment 4-3

- Write a function using conditional statements to evaluate the following function, assuming that the scalar variable  $x$  take a value from user . The function is

$$y = \sqrt{x^2 + 1}$$

$$y = 3x + 1$$

$$y = 9 \sin(5x - 50) + 31$$

- for  $x < 0$ ,
- for  $0 \leq x < 10$ ,
- for  $x \geq 10$

# Assignment 4-4

- If  $x = [5, -3, 18, 4]$  and  $y = [-9, 13, 7, 4]$ , what will be the result of the following operation ?
  - $A = y > x$
  - $B = y \& x$
  - $C = y | x$
  - $D = \text{find}(x == y)$
  - $E = (x > 3) \& (y < 7)$